CS-1101
PROGRAMMING FUNDAMENTALS

Lab 13

By

Engr. Muhammad Awais

Feb 18, 2025

Lab Contents

- Pointers

Lab Objectives

The objective of this lab is :

- To understand the concept of pointers in C++.
- To understand the concept of dereferencing operators.
- To get familiar with various types of pointers.
- To learn how to use pointer arithmetic.
- To learn how to use array name as a pointer.

Pointers

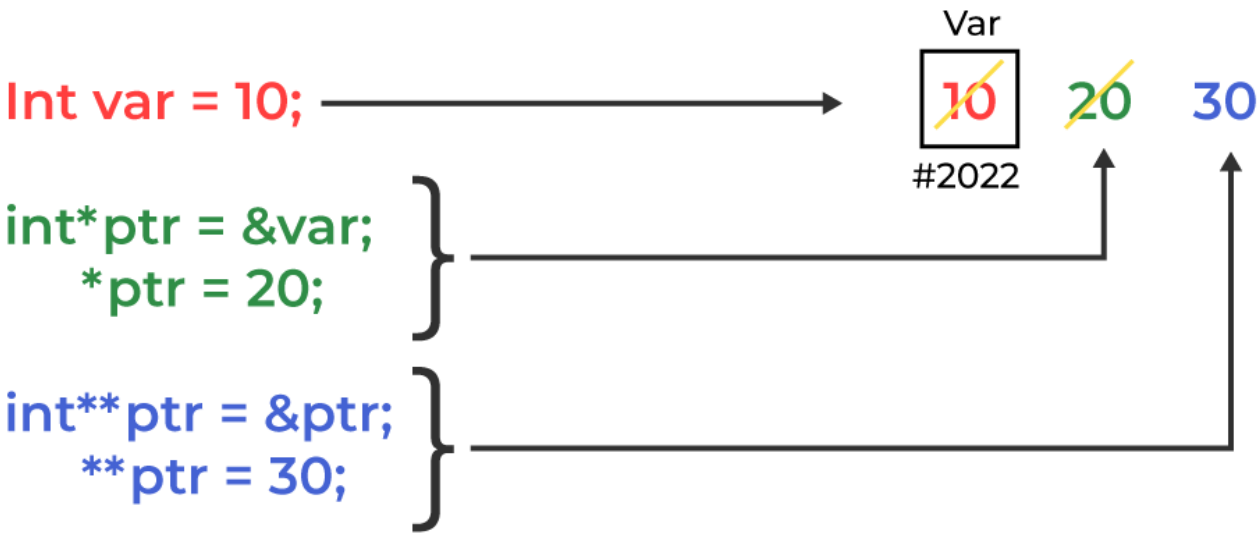
- Pointers are variables that store the memory addresses of other variables.
- Every variable we declare in our program has an associated location in the memory, which we call the memory address of the variable.
- If we have a variable var in our program, &var returns its memory address.

Syntax

```
datatype *var_name;
int *ptr;   // ptr can point to an address which holds int data
```

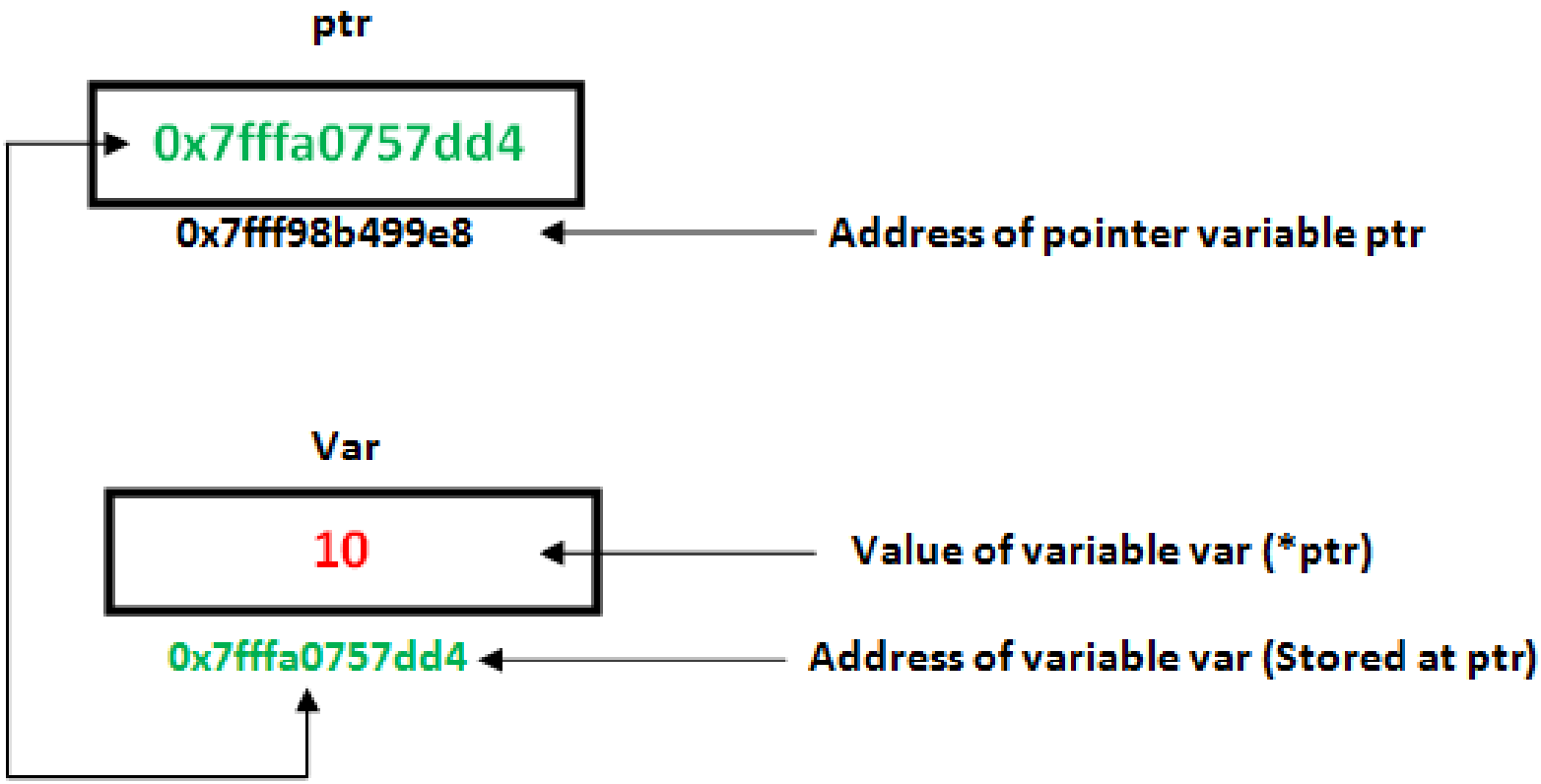
- The type of a pointer must match the type of the variable whose address it stores.

How Pointer Works in C++



- Define a pointer variable
- Assigning the address of a variable to a pointer using the unary operator (&) which returns the address of that variable.
- Accessing the value stored in the address using unary operator (*) which returns the value of the variable located at the address specified by its operand.

Cont…


ptr
0x7fffa0757dd4
0x7fff98b499e8
Address of pointer variable ptr
Var
10
Value of variable var (*ptr)
0x7fffa0757dd4
Address of variable var (Stored at ptr)

Types of a Pointer

Invalid pointers

- A pointer should point to a valid address but not necessarily to valid elements (like for arrays). These are called invalid pointers. Uninitialized pointers are also invalid pointers.

```
int *ptr1;
int arr[10];
int *ptr2 = arr+20;
```

- Here, **ptr1 is uninitialized** so it becomes an invalid pointer and **ptr2 is out of bounds of arr** so it also becomes an invalid pointer. (Note: invalid pointers do not necessarily raise compile errors)

Types of a Pointer

NULL pointers

- A null pointer is a pointer that point nowhere and not just an invalid address. Following are 2 methods to assign a pointer as NULL.

```
int *ptr1 = 0;
int *ptr2 = NULL;
```

Types of a Pointer

Void pointers

- This is a special type of pointer available in C++ which represents the absence of type.
- Void pointers are pointers that point to a value that has no type (undetermined dereferencing properties).
- This means that void pointers have great flexibility as they can point to any data type. There is a payoff for this flexibility.
- These pointers cannot be directly dereferenced. They have to be first transformed into some other pointer type that points to a concrete data type before being dereferenced.

```
void *ptr;  // Declare a void pointer

// Storing an integer address
ptr = &num;
```

```
*(int *)ptr   // If it points to an int
*(float *)ptr // If it points to a float
```

Void pointer Cont…

```
void *ptr;  // Declare a void pointer

// Storing an integer address
ptr = &num;
```

- Cannot be directly dereferenced → *(ptr) is invalid.
- Must be explicitly typecast before dereferencing

```
*(int *)ptr   // If it points to an int
*(float *)ptr // If it points to a float
```

Call-By-Reference with a Pointer Argument in C++

```
void modify(int *p) {
    *p = 20;  // Changing the value at the address
}

int x = 10;
modify(&x);  // Passing address of x
// Now, x = 20
```

Array Name as Pointers

- An array name contains the address of the first element of the array which acts like a constant pointer.
- It means, the address stored in the array name can't be changed. For example, if we have an array named **val then val and &val[0] can be used interchangeably.**

```
using namespace std;
void geeks()
{
    // Declare an array
    int val[3] = { 5, 10, 20 };

    // declare pointer variable
    int* ptr;

    // Assign the address of val[0] to ptr
    // We can use ptr=&val[0];(both are same)
    ptr = val;
    cout << "Elements of the array are: ";
    cout << ptr[0] << " " << ptr[1] << " " << ptr[2];
}
// Driver program
int main() { geeks(); }
```

Output

```
Elements of the array are: 5 10 20
```

Pointer Arithmetic

```cpp
// C++ program to illustrate Pointer Arithmetic
#include <bits/stdc++.h>
using namespace std;
void geeks()
{
    // Declare an array
    int v[3] = { 10, 100, 200 };

    // declare pointer variable
    int* ptr;

    // Assign the address of v[0] to ptr
    ptr = v;

    for (int i = 0; i < 3; i++) {
        cout << "Value at ptr = " << ptr << "\n";
        cout << "Value at *ptr = " << *ptr << "\n";

        // Increment pointer ptr by 1
        ptr++;
    }
}

// Driver program
int main() { geeks(); }
```

```
Value at ptr = 0x7ffe5a2d8060
Value at *ptr = 10
Value at ptr = 0x7ffe5a2d8064
Value at *ptr = 100
Value at ptr = 0x7ffe5a2d8068
Value at *ptr = 200
```

- Pointer arithmetic is meaningless unless performed on an array

Tasks

- Write a C++ program to declare an integer variable, initialize it with a value, and then declare a pointer to store its address. Use the pointer to display the value of the variable using dereferencing operator.
- Write a C++ program to declare an integer variable and a pointer pointing to it. Change the value of the variable using the pointer and display the modified value.
- Write a C++ program to declare an array of integers and use a pointer to access and display each element using pointer arithmetic.
- Write a C++ program that swaps two numbers using a function that takes pointers as arguments. Display the values before and after swapping.

Tasks

- Write a C++ program to declare a pointer and initialize it to nullptr. Check whether the pointer is null and display an appropriate message indicating that it does not point to any valid memory. If the pointer is not null, display the memory address it holds.
- Write a C++ program to declare a void pointer and assign it the address of an integer variable. Use the void pointer to access and display the value of the integer. Since a void pointer does not have a specific data type, cast it to the appropriate type before dereferencing

Home Task

- How can we use pointers for dynamic memory allocation?

References

- https://www.geeksforgeeks.org/cpp-pointers/